Primer Parcial
Examen Práctico

Prof. Carmen Lucia Bustillo Hernández

Universidad Nacional Autónoma de México

Fecha de Entrega: 9 de Abril de 2026

Nombre: Cortez Rios Enrique Yoab

No. cuenta: 317302992

Firma:

Nombre: Mendoza Gonzáles Mario

No. cuenta: 316284828

Firma:

Instrucciones

1. Lea su examen cuidadosamente. Ponga cuidado a los detalles.
2. Para la realización de su examen se requiere un equipo de dos personas.
3. La evaluación consiste en la demostración presencial del problema planteado, utilizando dos computadoras conectadas mediante cable de red.
4. Presentar pseudocódigo, diagrama de flujo y código respectivamente.
5. Presentar la documentación correspondiente a la base de datos.
6. El contenido que esté fuera de contexto del tema del examen será ANULADO.

Criterios de Evaluación

1. Modelo Entidad-Relación y Normalización de la Base de Datos = 12.5 pts.
2. Pseudocódigo y diagrama de flujo = 12.5 pts.
3. Código de la aplicación = 25 pts.

1. Problemas

1.1 Problema 1

Crear una aplicación (Utilizando sockets de tipo de flujo como datagrama y TCP) que desde la máquina cliente solicite a un servidor consultas sobre la base de datos (Universidad.xls). Normalice a la 4FN respectivamente la base de datos proporcionada. La aplicación debe consultar, modificar, eliminar o actualizar los registros de la base de datos respectivamente desde el cliente. Para el desarrollo de este problema, puede utilizar: **50 pts.**

- Un gestor de bases de datos (MySQL, PostgreSQL, entre otros).
- Gestión de la base de datos mediante archivos de texto.

Nota: Insertar todos los registros que vienen en el archivo Universidad.xls para realizar las consultas.

1.2 Solución

Para poder levantar la base de datos por medio de archivos de texto, lo principal es normalizar la base del archivo ”Universidad.xls”. Posterior a esto, debemos migrar los datos que contiene originalmente a la nueva estructura normalizada que se muestra a continuación:

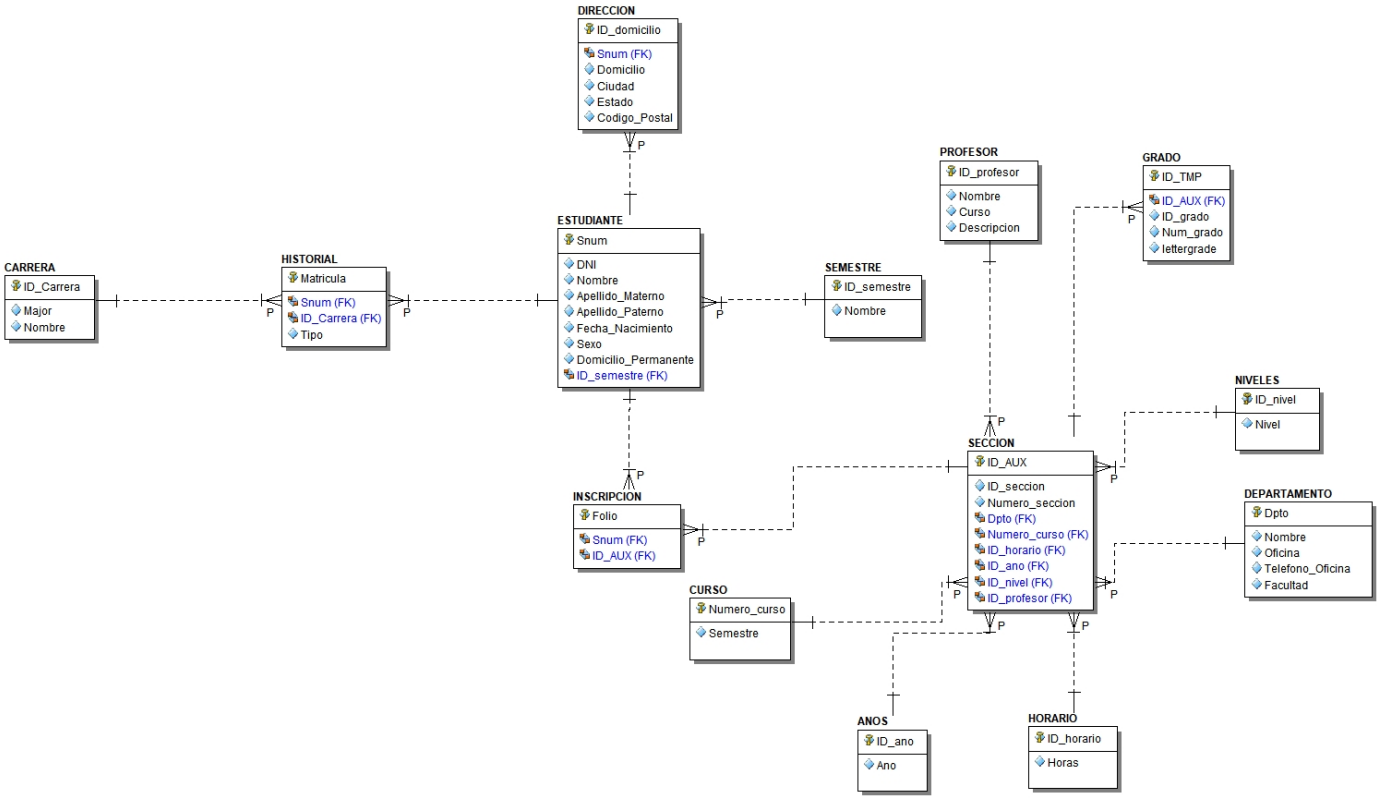

Figura 1: Diagrama Entidad-Relación

Con esto podemos identificar varios elementos importantes: son 14 tablas, las cuales debemos crear como archivos y llenarlas con sus respectivos datos.

1.2.1 Tabla Estudiantes

Esta tabla almacena los 20 registros de los estudiantes, su semestre y el domicilio permanente. Esto evita agregar registros vacíos dentro de la tabla dirección, además de ser atributos intrínsecos del estudiante.

```
You, hace 21 horas | 1 author (You)
Snum,DNI,Nombre,Apellido Materno,Apellido Paterno,Fecha,Sexo,Domicilio Permanente,Telefono Permanente,ID_semestre
S1001,521861420,María Luisa,Araceli,Vega,10/06/2005,M,Viaducto Arabia Saudita 216 Edif. 175 - Depto. 932,(695)036-3854x80667,7003
S1002,912416442,Carlota,Ariadna,Nájera,10/07/2003,F,Privada Solano 707 144,1-850-823-4521x14858,7002
S1003,777290831,Nicolás,Mitzy,Toledo,05/20/2005,F,Circunvalación Sonora 325 584,301-591-1557x61179,7000
S1004,179253543,Eric,Armando,Vargas,12/17/2003,F,Eje vial Correa 983 Interior 098,+08(7)4691764241,7004
S1005,82002031,María,Fidel,Pantoja,11/07/1996,F,Corredor Lovato 924 Interior 283,1-892-806-1324,7003
S1006,137616787,Carla,Octavio,Borrego,01/21/1997,M,Pasaje Castro 589 712,019.308.1976x12235,7002
S1007,296816353,Ángela,Espartaco,Aguilera,09/05/1998,F,Corredor Martínez 952 Interior 789,258.458.5416,7003
S1008,555203433,Esteban,Gabriela,Salazar,01/12/1995,F,Eje vial Veracruz de Ignacio de la Llave 694 Interior 889,1-504-167-2016x791,7003
S1010,309623581,Óscar,Lourdes,Estévez,12/05/2005,F,Calle Fuentes 606 Interior 272,1-124-477-4018,7003
S1011,131790292,María,Patricio,Olmos,01/18/1995,M,Peatonal Marrero 022 Edif. 212 - Depto. 808,103.545.4598,7000
S1012,60845126,Aurelio,Rodrigo,Arias,06/08/2001,M,Calzada Côte d'Ivoire 791 Interior 876,1-269-421-6148x9073,7003
S1013,319678542,Ramiro,Alejandro,Alcaraz,10/27/2002,F,Eje vial Barajas 894 Interior 386,1-687-708-4151x97961,7000
S1014,661794898,Aldonza,José Emilio,Barragán,10/05/1999,F,Avenida Sur del Río 747 Interior 900,1-584-050-6899x498,7002
S1015,559149988,Pedro,Beatriz,Longoria,07/05/2004,F,Avenida Sur de la Crúz 444 Interior 315,485-030-3120x63533,7002
S1016,504169559,Dolores,Abraham,Reyes,12/04/2006,F,Avenida Ortiz 654 Edif. 445 - Depto. 235,964.942.4955x25282,7004
S1017,828748077,Jorge,Natividad,Alemán,02/01/2001,M,Avenida Collado 957 Interior 947,086-049-7686x8322,7000
S1018,617296547,Abraham,Zacarías,Anguiano,05/09/2007,F,Callejón Sur Negrete 946 447,458.172.4786,7002
S1019,942739743,René,Ana María,Medina,07/08/2003,F,Pasaje Jalisco 963 Edif. 080 - Depto. 291,1-875-807-1294x257,7003
S1020,694465568,Elena,Alfredo,Zedillo,06/03/1995,F,Andador Norte Quiñones 354 Edif. 447 - Depto. 114,1-298-047-1825,7004
S1022,317302992,Enrique,Cortez,Rios,19/07/00,M,Avenida Aztecas,5622158556,7004
```

Tabla 1. Estudiante

1.2.2 Tabla Dirección

Esta tabla almacena la dirección actual del estudiante, la cual es más propensa a ser cambiada y por eso no se agrega directamente a la tabla de estudiantes (a diferencia de su domicilio permanente).

```
You, hace 4 días | 1 author (You)
ID_direccion,Snum,Domicilio,Ciudad,Estado,Codigo Postal
1,S1001,Boulevard Sur Niño 851 897,San Marco Antonio los bajos,Durango,63265
2,S1002,Privada Sur Olivo 683 843,San Cristal los altos,Querétaro,22487
3,S1003,Diagonal Sur García 485 832,San Anel los altos,Sonora,7110
4,S1004,Pasaje Ochoa 286 Interior 088,Nueva Países Bajos,Guerrero,41139-2439
5,S1005,Andador Puebla 544 Edif. 676 - Depto. 319,Vieja Samoa,Baja California,83058-9696
6,S1006,Periférico Valenzuela 027 Edif. 011 - Depto. 785,Vieja Uganda,Guerrero,04922-6067
7,S1007,Diagonal Segovia 385 932,San Rubén de la Montaña,Puebla,54226
8,S1008,Pasaje Norte Báez 158 681,San José Eduardo de la Montaña,Tabasco,33278-0324
9,S1009,Callejón Olmos 941 Edif. 256 - Depto. 634,San Rafaél los altos,Yucatán,82869
10,S1010,Avenida Santacruz 055 Interior 715,San Ana María de la Montaña,Nayarit,97199-7711
11,S1011,Calzada Lozada 232 720,Vieja Costa Rica,Colima,4354
12,S1012,Andador Querétaro 489 Interior 505,San Verónica los bajos,Guanajuato,86893-3995
13,S1013,Boulevard Vaca 238 Interior 252,Nueva Kazajstán,Baja California Sur,77245
14,S1014,Ampliación Brito 828 Edif. 517 - Depto. 268,Vieja Iraq,Jalisco,19683
15,S1015,Continuación Guillen 866 Interior 073,San Hermelinda de la Montaña,Tlaxcala,68371-4509
16,S1016,Prolongación Sur Apodaca 426 Interior 503,San Miguel los bajos,Distrito Federal,71535-4856
17,S1017,Circunvalación Distrito Federal 724 Edif. 338 - Depto. 251,Vieja República Dominicana,Quintana Roo,96509
18,S1018,Calzada Sur Moya 041 217,San Irene los bajos,Querétaro,62817
19,S1019,Retorno Hidalgo 828 Interior 858,San Virginia de la Montaña,Oaxaca,21284-0556
20,S1020,Andador Tayikistán 789 Interior 707,Vieja India,Chihuahua,87694
```

Tabla 2. Dirección

1.2.3 Tabla Carrera

La tabla carrera almacena los nombres de cada carrera. Debido a que un estudiante puede cursar una o más carreras, la relación es de muchos a muchos. Esto obliga a generar una tabla intermedia entre el estudiante y las carreras, que se llamará historial.

```csv
ID_carrera,Major,Carrera
101,ART,Licenciatura en Arquitectura
102,CSH,Licenciatura en Derecho
103,CSH,Licenciatura en Psicología
104,COM,Licenciatura en Comunicación
105,SAL,Licenciatura en Enfermería
106,ING,Ingeniería Industrial
107,ING,Ingeniería Mecánica
108,INF,Ingeniería en Sistemas Computacionales
109,ADM,Licenciatura en Administración
110,CNY,Licenciatura en Contaduría
111,EDU,NULL
112,COM,NULL
113,MAT,NULL
114,ING,NULL
115,CSH,NULL
```

Tabla 3. Carrera

1.2.4 Tabla Historial

Esta es la tabla intermedia que se genera entre el estudiante y el catálogo de carreras. Si un registro tiene más de una llave, significa que el alumno cursa más de una carrera.

```csv
Snum,Matricula,ID_carrera,Tipo
S1001,412501,101,PRIMERA
S1002,412502,102,PRIMERA
S1003,412503,104,PRIMERA
S1004,412504,103,PRIMERA
S1005,412505,105,PRIMERA
S1006,412506,105,PRIMERA
S1007,412507,106,PRIMERA
S1008,412508,107,PRIMERA
S1009,412509,105,PRIMERA
S1010,412510,101,PRIMERA
S1011,412511,108,PRIMERA
S1012,412512,102,PRIMERA
S1013,412513,109,PRIMERA
S1014,412514,110,PRIMERA
S1015,412515,105,PRIMERA
S1016,412516,107,PRIMERA
S1017,412517,101,PRIMERA
S1018,412518,107,PRIMERA
S1019,412519,108,PRIMERA
S1020,412520,110,PRIMERA
S1006,412521,111,SEGUNDA
S1007,412522,112,SEGUNDA
S1010,412523,105,SEGUNDA
S1013,412524,108,SEGUNDA
S1014,412525,104,SEGUNDA
S1018,412526,105,SEGUNDA
S1014,412527,106,TERCERA
S1018,412528,107,TERCERA
```

Tabla 4. Historial

1.2.5 Tabla Inscripción

Esta tabla es el producto de la relación entre el estudiante y la sección en la que está inscrito. Al ser una relación de muchos a muchos, la llamamos inscripción, pues un estudiante puede inscribir una o varias secciones.

```csv
ID_folio,Snum,ID_AUX
5000,S1001,501
5001,S1002,502
5002,S1003,503
5003,S1004,504
5004,S1005,505
5005,S1006,506
5006,S1007,507
5007,S1008,508
5008,S1009,509
5009,S1010,510
5010,S1011,511
5011,S1012,512
5012,S1013,513
5013,S1014,514
5015,S1016,515
5016,S1017,516
5017,S1018,517
5018,S1019,518
5019,S1020,519
```

Tabla 5. Inscripción

1.2.6 Tabla Sección

Esta tabla es la que almacena el registro de la sección inscrita, junto con el profesor y el departamento que la imparte, además de su calificación definida.

```csv
ID_AUX,Dpto,Numero Curso,ID_seccion,Numero de Seccion,ID_año,ID_horas,ID_nivel,ID_profesor
501,EDU,EDU-201,SEC21,1,1500,3000,2000,4000
502,INF,ING-301,SEC4,2,1501,3000,2001,4001
503,MAT,MAT-101,SEC23,2,1500,3001,2002,4002
504,INF,ING-401,SEC5,1,1501,3001,2002,4003
505,EDU,EDU-101,SEC20,1,1501,3000,2000,4004
506,SAL,SAL-101,SEC14,1,1500,3000,2001,4005
507,EDU,EDU-201,SEC21,1,1500,3000,2002,4000
508,ADM,ADM-101,SEC6,3,1500,3000,2001,4006
509,SAL,SAL-101,SEC14,1,1500,3000,2002,4005
510,COM,COM-201,SEC26,3,1502,3001,2000,4007
511,EDU,EDU-201,SEC21,1,1500,3000,2001,4000
512,SAL,SAL-101,SEC14,1,1500,3000,2000,4005
513,ADM,ADM-301,SEC10,3,1501,3000,2002,4008
514,SAL,SAL-101,SEC14,1,1500,3000,2000,4005
515,ART,ART-201,SEC18,1,1500,3000,2000,4009
516,EDU,EDU-201,SEC21,1,1500,3000,2001,4000
517,ADM,ADM-101,SEC6,3,1500,3000,2002,4006
518,ADM,ADM-101,SEC7,2,1501,3000,2001,4010
519,ADM,ADM-101,SEC7,2,1501,3000,2000,4010
```

Tabla 6. Sección

1.2.7 Tabla Profesor

La tabla profesor almacena el nombre, el curso y una breve descripción de la asignatura que imparte el docente en una o varias secciones.

```
ID_profesor,Profesor,Nombre Curso,Descripción
4000,Diana Romero,Diseño de Material Didáctico,Planeación y diseño de materiales.
4001,Rufino Valentín Aguirre,Bases de Datos,Modelo relacional y SQL.
4002,Tania Franco,Cálculo Diferencial,Límites y derivadas.
4003,Asunción Guillermo Alemán,Redes de Computadoras,Fundamentos de redes.
4004,Sofía Quiñónez,Teorías del Aprendizaje,Modelos de aprendizaje.
4005,Horacio Ponce,Anatomía Humana,Estructura del cuerpo humano.
4006,Alvaro Ortiz,Fundamentos de Administración,Principios básicos de administración.
4007,Margarita Cristian Aparicio,Periodismo Digital,Producción de contenidos digitales.
4008,Luis Manuel Barbara Parra,Finanzas Corporativas,Análisis financiero y decisiones de inversión.
4009,Estela Berta Calvillo,Teoría de la Arquitectura,Historia y teoría de la arquitectura.
4010,Claudio Hermelinda Puente,Fundamentos de Administración,Principios básicos de administración.
```

Tabla 7. Profesor

1.2.8 Tabla Departamento

El departamento contiene los nombres de cada uno, su abreviatura, un teléfono de oficina y la facultad a la que pertenece la sección.

```
Depto,Nombre,Oficina,Telefono_Oficina,Facultad
EDU,Departamento de Educación,Edificio 5 Oficina 271,1-798-800-8189x1975,Facultad de Ciencias de la Educación
INF,Departamento de Informática,Edificio 2 Oficina 170,(454)064-2378x36744,Facultad de Ingeniería
MAT,Departamento de Matemáticas,Edificio 3 Oficina 388,1-757-934-6909x675,Facultad de Ciencias
SAL,Departamento de Ciencias de la Salud,Edificio 1 Oficina 120,1-964-894-2395x750,Facultad de Medicina
ADM,Departamento de Administración,Edificio 1 Oficina 494,888.204.5266x03624,Facultad de Contaduría y Administración
COM,Departamento de Comunicación,Edificio 3 Oficina 460,1-448-272-3458,Facultad de Ciencias de la Comunicación
ART,Departamento de Arte y Diseño,Edificio 4 Oficina 375,1-337-227-9541,Facultad de Arquitectura y Diseño
```

Tabla 8. Departamento

1.2.9 Tabla Niveles

Esta tabla solo almacena el nivel en el que se impartió la sección.

```
ID_nivel,Nivel
2000,Licenciatura
2001,Posgrado
2002,Doctorado
```

Tabla 9. Niveles

1.2.10 Tabla Horario

La tabla guarda el número de horas en las que se imparte una sección.

```
ID_horas,Horas
3000,4
3001,5
```

Tabla 10. Horario

1.2.11 Tabla Grado

La tabla grado almacena la calificación obtenida en una sección.

```
ID_TMP,ID_AUX,ID_grado,Num_grado,Lettergrade
6001,501,G120,61,D
6002,502,G10,98,A
6003,503,G17,60,D
6004,504,G68,95,A
6005,505,G121,69,D
6006,506,G14,60,D
6007,507,G110,96,A
6008,508,G179,78,C
6009,509,G192,98,A
6010,510,G69,67,D
6011,511,G101,70,C
6012,512,G132,70,C
6013,513,G130,64,D
6014,514,G40,70,C
6015,515,G33,73,C
6016,516,G115,79,C
6017,517,G89,60,D
6018,518,G59,76,C
6019,519,G85,82,B
```

Tabla 11. Grado

1.2.12 Tabla Curso

La tabla curso almacena la estación del año en la que se imparte la sección y el salón asignado.

```
Número_Curso,Semestre
EDU-101,Verano
EDU-201,Otoño
ING-301,Primavera
ING-401,Primavera
MAT-101,Primavera
SAL-101,Primavera
COM-201,Otoño
ADM-101,Otoño
ADM-301,Otoño
ART-201,Otoño
```

Tabla 12. Curso

1.2.13 Tabla Años

La tabla años solo almacena el año en el que se impartió la sección.

```
ID_año,Año
1500,2023
1501,2024
1502,2025
```

Tabla 13. Años

1.2.14 Tabla Semestre

Esta tabla solo almacena el semestre en el que se encuentra el estudiante.

```
ID_semestre,Nombre
7000,1er semestre
7002,3er semestre
7003,5to smestre
7004,7mo semestre
```

Tabla 14. Semestre

- **Estructura del Cliente**

El cliente está diseñado bajo el patrón arquitectónico de **Cliente Ligero** (*Thin Client*). Carece de lógica de negocio o rutinas de verificación locales; delega el 100% del procesamiento transaccional y la validación de integridad al servidor, el cual se encarga de reportar cualquier anomalía. Para asegurar una interacción exitosa con la base de datos, el cliente se encarga exclusivamente de sincronizar y empaquetar las peticiones siguiendo el protocolo y los parámetros exigidos por el servidor.
Para establecer la interacción bidireccional, se implementó una interfaz de línea de comandos (CLI) que presenta un menú interactivo con las operaciones disponibles. La captura de instrucciones se procesa mediante el análisis de cadenas de texto (*parsing*), utilizando funciones de tokenización para enrutar la petición del usuario hacia el submódulo correspondiente.

Código 1. Cliente TCP (MAIN)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>

#define IP_SERVIDOR "192.168.1.20" // Esta es la ip del servidor
```

```c
#define PUERTO 3000
#define TAM_MAX 1024

void realizar_consulta(int sock);
void realizar_insercion(int sock);
void realizar_eliminacion(int sock);
void realizar_actualizacion(int sock);

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[TAM_MAX];

    printf("[1] Creando socket TCP...\n");
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) return -1;
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PUERTO);
    if (inet_pton(AF_INET, IP_SERVIDOR, &serv_addr.sin_addr) <= 0) return -1;

    printf("[2] Conectando al servidor...\n");
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) return -1;

    bzero(buffer, TAM_MAX);
    recv(sock, buffer, TAM_MAX, 0);
    printf("\n[Mensaje Inicial del Servidor]: %s", buffer);

    // --- BUCLE INTERACTIVO ---
    while(1) {
        printf("\n ==========================================================================\n");
        printf(" COMANDOS ACTIVOS: CONSULTA | INSERCION | ELIMINACION | ACTUALIZACION | OFF |\n");
        printf(" ==========================================================================\n");
        printf("Tu mensaje (Comando): ");

        bzero(buffer, TAM_MAX);
        fgets(buffer, TAM_MAX, stdin);
        buffer[strcspn(buffer, "\n")] = 0;

        if (strstr(buffer, "OFF") != NULL) {
            printf("[!] Cerrando cliente...\n");
            send(sock, "OFF", 3, 0);
            break;
        }
        else if (strstr(buffer, "CONSULTA") != NULL) {
            realizar_consulta(sock);
        }
```

```
        else if(strstr(buffer, "INSERCION") != NULL)
        {
            realizar_insercion(sock);
        }
        else if(strstr(buffer, "ELIMINACION") != NULL)
        {
            realizar_eliminacion(sock);
        }
        else if(strstr(buffer, "ACTUALIZACION") != NULL)
        {
            realizar_actualizacion(sock);
        }else {
            printf("[-] Comando no reconocido. Escribe: CONSULTA, INSERCION, ACTUALIZACION, ELIMINACION o OFF.\n");
        }
    }

    close(sock);
    printf("\nDesconectado exitosamente.\n");
    return 0;
}
```

Se desarrolla la función consulta, que es una de las operaciones más usadas dentro de una base de datos. Para que esta sea procesada por el servidor, el cliente debe enviar 3 cosas:

- **Número de tabla:** El identificador de la tabla que se consulta dentro de la base.
- **Llave:** Es el tipo de consulta que se realiza (total, parcial o único).
  1. **full:** Revisará toda la tabla y la regresa completa.
  2. **low:** Revisa toda la tabla pero retorna solo los atributos que escoja el cliente.
  3. **ID:** Es el identificador único de cada registro. Si existe, regresa la consulta con los parámetros que define visualizar el cliente.
- **Parámetros:** Es una cadena de bits que indica qué atributos de la tabla consultada se desean observar ('1' es para visualizarlo y '0' es para ignorarlo). Por lo tanto, esta cadena de texto es diferente para cada tabla que se encuentra dentro de la base de datos.

Al obtener estos datos por medio del teclado, debemos concatenar las respuestas con una barra vertical "—" (o el separador definido) para que el servidor pueda descomponerlo después y verificar si la petición es correcta. En caso de que falle algún dato o parámetro, el mismo servidor regresará el error producido; de lo contrario, se entra en un bucle de comunicación para poder transferir los datos del servidor al cliente. El cliente tendrá un archivo llamado `consulta.csv` en el que guardará la información obtenida (el contenido se sobrescribirá cada vez que se haga una nueva consulta).

Código 2. Cliente TCP (CONSULTA)

```
void realizar_consulta(int sock) {
```

```
    char tabla[10], llave[50], mascara[50], paquete_envio[TAM_MAX], respuesta_servidor[TAM_MAX];
    printf("\n\t--- MODO CONSULTA ---\n");
    printf("[ESTUDIANTE,0]\t\t[DIRECCION,1]\n[CARRERA,2]\t\t[HISTORIAL,3]\n[INSCRIPCION,4]\t\t[SECCION,5]\n[PROFESOR,6]\t\t[DEPARTAMENTO,7]\n");
    printf("[NIVELES,8]\t\t[HORARIO,9]\n[GRADO,10]\t\t[CURSO,11]\n[ANOS,12]\t\t[SEMESTRE,13]\n");
    printf("Ingresa el numero de tabla (0-13): ");
    fgets(tabla, sizeof(tabla), stdin); tabla[strcspn(tabla, "\n")] = 0;
    printf("Ingresa la llave a buscar ('full', 'low' o ID): ");
    fgets(llave, sizeof(llave), stdin); llave[strcspn(llave, "\n")] = 0;
    printf("Ingresa la mascara de parametros (Bits 1/0): ");
    fgets(mascara, sizeof(mascara), stdin); mascara[strcspn(mascara, "\n")] = 0;

    sprintf(paquete_envio, "1|%s|%s|%s", tabla, llave, mascara);
    send(sock, paquete_envio, strlen(paquete_envio), 0);

    // --- NUEVA LOGICA DE RECEPCION ---
    bzero(respuesta_servidor, sizeof(respuesta_servidor));
    recv(sock, respuesta_servidor, sizeof(respuesta_servidor), 0);
    // Verificamos si el servidor activó el modo de envío por renglones
    if (strncmp(respuesta_servidor, "CANTIDAD|", 9) == 0) {
        // Extraemos el número que viene después de "CANTIDAD|"
        int total_filas = atoi(respuesta_servidor + 9);
        if(strstr(llave, "full") != NULL || strstr(llave, "FULL") != NULL || strstr(llave, "low") != NULL || strstr(llave, "LOW") != NULL ) total_filas--;
        printf("\n--- RESULTADOS DE LA BD (%d registros) ---\n", total_filas);
        // Abrimos el archivo en donde se guardara la consulta
        FILE *archivo = fopen("./consulta.csv", "w");
        // Le decimos al servidor: "Estoy listo, manda el primero" (ACK)
        send(sock, "ACK", 3, 0);
        // Bucle para recibir uno por uno
        for(int i = 0; i < total_filas; i++) {
            bzero(respuesta_servidor, sizeof(respuesta_servidor));
            recv(sock, respuesta_servidor, sizeof(respuesta_servidor), 0);

            // Imprimimos el renglón recibido
            printf("[%d]: %s\n", i, respuesta_servidor);
            fprintf(archivo,"%s\n",respuesta_servidor);
            // Confirmamos recepción (ACK) para que el servidor mande el siguiente
            send(sock, "ACK", 3, 0);
        }
        printf("----------------------------------------\n");
        fclose(archivo);
    } else {
        // Si no era "CANTIDAD|", significa que es un Error o un "No se encontraron registros"
        printf("\n[Servidor]: %s\n", respuesta_servidor);
```

```
    }
}
```

La función de inserción solicita solo 1 parámetro inicial para enviar al servidor, el cual retornará la cantidad de columnas o registros que se deben llenar para poder ser insertados en la tabla. El mismo servidor verifica que todos los campos estén completos (de lo contrario no podrá insertarlo), además de validar que la llave primaria ingresada no se repita en la tabla. Si el servidor retorna un valor válido, empieza el proceso de captura de los datos solicitados.

Código 3. Cliente TCP (INSERCION)

```c
void realizar_insercion(int sock)
{
    char tabla[10], paquete_envio[TAM_MAX], respuesta_servidor[TAM_MAX];
    char datos_csv[TAM_MAX] = "";
    // Aquí armaremos la fila con comas
    char entrada_temporal[256];
    //El buffer que almacenara lo del teclado

    printf("\n--- MODO INSERCION ---\n");
    printf("Ingresa el numero de tabla (0-13): ");
    fgets(tabla, sizeof(tabla), stdin); tabla[strcspn(tabla, "\n")] = 0;
    // FASE 1: Enviar petición inicial al servidor (Operación 2 | Tabla)
    sprintf(paquete_envio, "2|%s", tabla);
    send(sock, paquete_envio, strlen(paquete_envio), 0);

    // FASE 2: Esperar respuesta del servidor (Debería ser "COLUMNAS|N")
    bzero(respuesta_servidor, sizeof(respuesta_servidor));
    recv(sock, respuesta_servidor, sizeof(respuesta_servidor), 0);

    if (strncmp(respuesta_servidor, "COLUMNAS|", 9) == 0) {

        // Extraemos el número de columnas que el servidor contó
        int num_columnas = atoi(respuesta_servidor + 9);
        printf("[*] El servidor verifico la tabla. Se requieren %d columnas.\n",
    num_columnas);
        printf("--------------------------------------------------\n");
        // Preparamos el inicio del paquete final
        strcpy(datos_csv, "DATOS|");
        // FASE 3: Bucle interactivo pidiendo al usuario dato por dato
        for (int i = 0; i < num_columnas; i++) {
            if (i == 0) {
                printf("Ingrese el dato para la columna %d (LLAVE PRIMARIA): ", i + 1);
            } else {
                printf("Ingrese el dato para la columna %d: ", i + 1);
            }

            bzero(entrada_temporal, sizeof(entrada_temporal));
            fgets(entrada_temporal, sizeof(entrada_temporal), stdin);
```

```c
            entrada_temporal[strcspn(entrada_temporal, "\n")] = 0; // Quitar salto

            // Concatenamos el dato ingresado a nuestra cadena principal
            strcat(datos_csv, entrada_temporal);
            // Si no es la última columna, le agregamos una coma separadora
            if (i < num_columnas - 1) {
                strcat(datos_csv, ",");
            }
        }

        // FASE 4: Enviar la fila completa al servidor
        printf("\n[Enviando paquete]: %s\n", datos_csv);
        send(sock, datos_csv, strlen(datos_csv), 0);

        // FASE 5: Esperar el veredicto final (Exito o Error por llave duplicada)
        bzero(respuesta_servidor, sizeof(respuesta_servidor));
        recv(sock, respuesta_servidor, sizeof(respuesta_servidor), 0);

        printf("[Respuesta del Servidor]: %s\n", respuesta_servidor);

    } else {
        // Si el servidor no respondió "COLUMNAS|", significa que hubo un error (ej. tabla inválida)
        printf("\n[Servidor rechazo la peticion]: %s\n", respuesta_servidor);
    }
}
```

Para la función de eliminación, el cliente solo necesita capturar 2 datos:

1. **Número de Tabla:** El identificador de la tabla donde se eliminará el registro.
2. **ID único:** La llave primaria del registro en la base de datos.

Código 4. Cliente TCP (ELIMINACION)

```c
void realizar_eliminacion(int sock)
{
    char tabla[10], llave[50], paquete_envio[TAM_MAX], respuesta_servidor[TAM_MAX];
    printf("\n--- MODO ELIMINACION ---\n");
    printf("Ingresa el numero de tabla (0-13): ");
    fgets(tabla, sizeof(tabla), stdin); tabla[strcspn(tabla, "\n")] = 0;
    printf("Ingresa el ID (llave primaria) a eliminar: ");
    fgets(llave, sizeof(llave), stdin); llave[strcspn(llave, "\n")] = 0;
    // FASE 1: Enviamos Operacion(3) | Tabla | Llave
    sprintf(paquete_envio, "3|%s|%s", tabla, llave);
    send(sock, paquete_envio, strlen(paquete_envio), 0);
    // FASE 2: Esperamos la respuesta del servidor
    bzero(respuesta_servidor, TAM_MAX);
    recv(sock, respuesta_servidor, TAM_MAX, 0);
```

```
    printf("\n[Respuesta del Servidor]: %s\n", respuesta_servidor);
}
```

La función de actualización debe capturar nuevamente lo siguiente:

1. **Número de Tabla:** La tabla en la que se actualizará un registro.
2. **ID:** El identificador único del registro que se va a modificar.

Al obtener esto del usuario, se envía la llave para revisar si existe en esa tabla antes de intentar actualizarlo. La respuesta del servidor indicará el número de columnas que contiene esa tabla; de lo contrario, si la llave no existe, no se podrá actualizar el registro.

Código 5. Cliente TCP (ACTUALIZACION)

```
void realizar_actualizacion(int sock) {
    char tabla[10], llave_vieja[50], paquete_envio[TAM_MAX], respuesta_servidor[TAM_MAX
    ];
    char datos_csv[TAM_MAX] = "";
    char entrada_temporal[256];

    printf("\n--- MODO ACTUALIZACION ---\n");
    printf("Ingresa el numero de tabla (0-13): ");
    fgets(tabla, sizeof(tabla), stdin);
    tabla[strcspn(tabla, "\n")] = 0;

    printf("Ingresa la llave ORIGINAL del registro a modificar: ");
    fgets(llave_vieja, sizeof(llave_vieja), stdin); llave_vieja[strcspn(llave_vieja, "\n
    ")] = 0;
    // FASE 1: Le enviamos la tabla y la llave vieja al servidor para ver si existe
    sprintf(paquete_envio, "4|%s|%s", tabla, llave_vieja);
    send(sock, paquete_envio, strlen(paquete_envio), 0);

    bzero(respuesta_servidor, TAM_MAX);
    recv(sock, respuesta_servidor, TAM_MAX, 0);
    // FASE 2: Si el servidor responde con las columnas, significa que el registro SI
    existe
    if (strncmp(respuesta_servidor, "COLUMNAS|", 9) == 0) {

        int num_columnas = atoi(respuesta_servidor + 9);
        printf("[*] Registro encontrado. Ingresa los nuevos datos (%d columnas).\n",
    num_columnas);
        printf("--------------------------------------------------\n");

        strcpy(datos_csv, "DATOS|");
        // FASE 3: Bucle interactivo para armar la nueva fila
        for (int i = 0; i < num_columnas; i++) {
            if (i == 0) {
                printf("NUEVA Llave Primaria (Puede ser la misma '%s'): ", llave_vieja);
            } else {
```

```
                printf("NUEVO dato para la columna %d: ", i + 1);
            }

            bzero(entrada_temporal, sizeof(entrada_temporal));
            fgets(entrada_temporal, sizeof(entrada_temporal), stdin);
            entrada_temporal[strcspn(entrada_temporal, "\n")] = 0;

            strcat(datos_csv, entrada_temporal);
            if (i < num_columnas - 1) strcat(datos_csv, ",");
        }

        // FASE 4: Enviar la fila completa
        printf("\n[Enviando actualizacion]: %s\n", datos_csv);
        send(sock, datos_csv, strlen(datos_csv), 0);

        // FASE 5: Recibir éxito o error (ej. si la nueva llave choca con otra)
        bzero(respuesta_servidor, TAM_MAX);
        recv(sock, respuesta_servidor, TAM_MAX, 0);
        printf("[Respuesta del Servidor]: %s\n", respuesta_servidor);

    } else {
        // Si no respondió "COLUMNAS|", es un error (seguramente la llave no existe)
        printf("\n[Servidor rechazo la peticion]: %s\n", respuesta_servidor);
    }
}
```

Con esto definido por parte del cliente, el servidor debe solucionar la comunicación entrante, la lectura de los archivos y la identificación de las llaves dentro de las tablas, además de informar sobre los errores internos que no permiten completar la petición del cliente.

- **Estructura del Servidor (`servidor.c`)**

El servidor representa el núcleo de procesamiento de este proyecto. Su labor no solo consiste en recibir y discretizar las cadenas de texto enviadas por el cliente, sino en mantener el código estrictamente modulado. Dado que concentrar toda la lógica en un solo archivo es una mala práctica de diseño, se tomó la decisión arquitectónica de generar archivos de cabecera e implementación independientes para cada una de las cuatro operaciones principales (CONSULTA, INSERCIÓN, ELIMINACIÓN y ACTUALIZACIÓN).
El archivo principal, `servidor.c`, actúa exclusivamente como el orquestador o controlador de tráfico del sistema, dividiendo su ejecución en tres fases críticas:

1. **Inicialización Concurrente (Multihilo)**

Antes de iniciar la conexión con el cliente, el servidor debe preparar su entorno. Para minimizar el tiempo de arranque, el sistema emplea la biblioteca `pthread.h` para lanzar dos hilos de ejecución en paralelo:

- **Hilo de Directorios (`funcion_h1`):** Se encarga de construir dinámicamente y almacenar en memoria todas las rutas absolutas hacia los archivos físicos (`.csv`) y hacia el entorno de *buffer* (archivos temporales), los cuales son fundamentales para las operaciones transaccionales.
- **Hilo de Red (`funcion_h2`):** Simultáneamente, interactúa con el sistema operativo para instanciar el socket TCP, realizar el *bind* al puerto 3000 y poner el servicio en modo de escucha.

El hilo principal espera a que ambos procesos concluyan mediante barreras de sincronización que nos ofrecen las liberias (`pthread_join`) antes de proceder a aceptar clientes.

2. Ciclo de Escucha y Desempaquetado

Una vez establecida la sesión con el cliente, el servidor entra en un bucle de escucha activa. Cuando recibe una trama de datos, utiliza funciones de manipulación de cadenas (`strtok`) para extraer la cabecera del mensaje utilizando la barra vertical (`|`) como carácter delimitador de protocolo. El primer segmento de la trama siempre indica la operación a realizar (1=Consulta, 2=Inserción, 3=Eliminación, 4=Actualización), permitiendo al servidor enrutar la petición correctamente.

3. Enrutador de Operaciones (Mini-Switch)

Dependiendo del código de operación recibido, el servidor delega la carga de trabajo a los módulos correspondientes:

- **Consultas e Eliminaciones:** Extrae la llave primaria de la trama y transfiere el control a las funciones `procesar_consulta` o `solicitud_eliminacion`, las cuales resuelven la petición y devuelven el resultado.
- **Inserciones y Actualizaciones (Descubrimiento de Esquema):** Para estas operaciones, el servidor implementa un protocolo interactivo en dos fases. Primero, intercepta la petición, invoca al analizador de archivos físicos para descubrir cuántas columnas tiene la tabla seleccionada, y le informa este número al cliente mediante la etiqueta `COLUMNAS|N`. Posteriormente, se queda a la espera de un segundo paquete (etiquetado como `DATOS|`) con la fila completa preformateada por el cliente, la cual finalmente se inyecta en las rutinas de modificación física.

Además de coordinar el flujo, el servidor implementa rutinas de seguridad de bajo nivel, como la clonación de llaves primarias en variables de respaldo (*backup*) durante las actualizaciones, evitando así errores críticos de segmentación o punteros huérfanos provocados por la sobreescritura de los *buffers* de red.

Código 1. Servidor TCP (MAIN)

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include "./extensiones/entidades.h"
#include "./extensiones/consulta.h"
```

```c
#include "./extensiones/insercion.h"
#include "./extensiones/update.h"

#define PUERTO 3000

// Variables globales
ARCHIVERO directorio;
char *escritura = "./buffer/";
char *carpeta = "./Datos/";
char *extension = ".csv";
char *archivos[] = {"Estudiante", "Direccion", "Carrera", "Historial", "Inscripcion", "
    Seccion", "Profesor", "Departamento", "Niveles", "Horario", "Grado", "Curso", "Años",
     "Semestre"};
int tamano = sizeof(archivos) / sizeof(archivos[0]);
int TAM_MAX = 1024;

// Prototipos de funciones
void* funcion_h1(void *arg);
void* funcion_h2(void *arg);
void procesar_consulta(int socket_cliente, int tabla, char *llave, char *mascara,
    ARCHIVERO *dir);

int main(void){
    // 1. INICIALIZACION DEL DIRECTORIO Y LA CONEXION TCP CON HILOS
    pthread_t hilo_directorio, hilo_red;
    HILO_DIR datos_dir = {&directorio, carpeta, escritura, archivos, extension, tamano};
    HILO_RED datos_red = {PUERTO, -1};

    printf("[Servidor] Iniciando hilos de preparacion...\n");
    pthread_create(&hilo_directorio, NULL, funcion_h1, (void *)&datos_dir);
    pthread_create(&hilo_red, NULL, funcion_h2, (void *)&datos_red);

    pthread_join(hilo_directorio, NULL);
    pthread_join(hilo_red, NULL);

    int server_fd = datos_red.socket_fd_resultado;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    int nuevo_socket;
    char buffer[TAM_MAX];

    printf("\n[+] Servidor iniciado. Esperando conexion...\n");

    nuevo_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);
    if (nuevo_socket < 0) {
        perror("ERROR: Falla al intentar conectar");
        close(server_fd);
        return -1;
```

```
    }

    printf("[+] Cliente conectado. Iniciando sesion única.\n");
    send(nuevo_socket, "Conexion establecida, el servidor finalizara despues de la sesion.\n", 67, 0);

    // 2. LOOP DE INTERACCION CON EL CLIENTE
    while(1) {
        bzero(buffer, TAM_MAX);
        int bytes_leidos = recv(nuevo_socket, buffer, TAM_MAX, 0);

        if (bytes_leidos <= 0 || strcasecmp(buffer, "OFF") == 0) {
            printf("[-] Cliente desconectado o se recibio comando OFF. Finalizando...\n");
            break;
        }

        printf("\n[Peticion Recibida]: %s\n", buffer);

        // Desempaquetamos los encabezados de la peticion
        char *op_str = strtok(buffer, "|");
        if (op_str == NULL) continue;
        int operacion = atoi(op_str);

        char *tabla_str = strtok(NULL, "|");
        if (tabla_str == NULL) continue;
        int tabla = atoi(tabla_str);

        // ==========================================
        // OPERACION 1: CONSULTA
        // ==========================================
        if (operacion == 1) {
            char *llave = strtok(NULL, "|");
            char *mascara = strtok(NULL, "");

            // Delegamos todo el trabajo a la función
            procesar_consulta(nuevo_socket, tabla, llave, mascara, &directorio);
        }
        // ==========================================
        // OPERACION 2: INSERCION
        // ==========================================
        else if(operacion == 2)
        {
            // FASE 1: Descubrimiento de esquema (Columnas)
            char *ruta_archivo = directorio.rutas[tabla];
            ANALISIS_ARCHIVO analisis = analizar_archivo(ruta_archivo);

            char msj_columnas[50];
```

```c
            // Le enviamos al cliente la etiqueta COLUMNAS| seguida del número
            sprintf(msj_columnas, "COLUMNAS|%d", analisis.num_columnas);
            send(nuevo_socket, msj_columnas, strlen(msj_columnas), 0);

            // FASE 2: Esperamos recibir los datos empaquetados por el cliente
            bzero(buffer, TAM_MAX);
            int bytes_datos = recv(nuevo_socket, buffer, TAM_MAX, 0);

            if (bytes_datos > 0 && strncmp(buffer, "DATOS|", 6) == 0) {
                // Brincamos los primeros 6 caracteres ("DATOS|") para obtener solo el CSV
                char *datos_csv = buffer + 6;

                // Preparamos la estructura
                INSERCION peticion_ins;
                peticion_ins.numero_tabla = tabla;
                peticion_ins.datos = datos_csv;
                peticion_ins.error = NULL;

                // Delegamos al motor físico (el que programamos en insercion.c)
                solicitud_insercion(&peticion_ins, &directorio);

                // FASE 3: Respondemos el resultado final (EXITO o ERROR de llave duplicada)
                if (peticion_ins.error != NULL) {
                    send(nuevo_socket, peticion_ins.error, strlen(peticion_ins.error), 0);
                    free(peticion_ins.error); // IMPORTANTE: Liberar el strdup
                } else {
                    char *err_desconocido = "ERROR: Fallo desconocido en insercion.\n";
                    send(nuevo_socket, err_desconocido, strlen(err_desconocido), 0);
                }
            } else {
                // Si el cliente mandó basura en lugar de "DATOS|..."
                char *err_proto = "ERROR: Se rompio el protocolo de insercion.\n";
                send(nuevo_socket, err_proto, strlen(err_proto), 0);
            }
        }
        // ==========================================
        // OPERACION 3: ELIMINACION
        // ==========================================
        else if (operacion == 3) {
            char *llave_eliminar = strtok(NULL, ""); // Extraemos la llave

            if (llave_eliminar != NULL) {
                ELIMINACION peticion_del;
                peticion_del.num_tabla = tabla;
                peticion_del.primary_key = llave_eliminar;
```

```
                peticion_del.error = NULL;

                solicitud_eliminacion(&peticion_del, &directorio);

                if (peticion_del.error != NULL) {
                    send(nuevo_socket, peticion_del.error, strlen(peticion_del.error), 0);
                    free(peticion_del.error);
                } else {
                    char *err = "ERROR: Fallo desconocido en eliminacion.\n";
                    send(nuevo_socket, err, strlen(err), 0);
                }
            }
        }
        // ==========================================
        // OPERACION 4: ACTUALIZACION
        // ==========================================
        else if (operacion == 4) {
            char *llave_vieja = strtok(NULL, "");

            // FASE 1: Verificamos que la llave original EXISTA en la tabla
            char *ruta_archivo = directorio.rutas[tabla];
            char *estado = validar_llave(llave_vieja, tabla, ruta_archivo);

            if (strcmp(estado, "EXITO") == 0) {
                // EXITO en validar_llave significa que la llave NO existe.
                char *msj = "ERROR: El registro que intentas actualizar no existe.\n";
                send(nuevo_socket, msj, strlen(msj), 0);
                continue; // Cancelamos la operación
            }

            // *** LA CURA AL BUG: RESPALDAR LA LLAVE ***
            // Hacemos una copia profunda de la llave para que no se destruya al limpiar el buffer
            char llave_respaldo[50];
            strcpy(llave_respaldo, llave_vieja);

            // FASE 2: Si existe, le decimos al cliente cuántas columnas necesitamos
            ANALISIS_ARCHIVO analisis = analizar_archivo(ruta_archivo);
            char msj_columnas[50];
            sprintf(msj_columnas, "COLUMNAS|%d", analisis.num_columnas);
            send(nuevo_socket, msj_columnas, strlen(msj_columnas), 0);

            // FASE 3: Esperamos los datos nuevos
            bzero(buffer, TAM_MAX); // Aquí se destruye el contenido original
            int bytes_datos = recv(nuevo_socket, buffer, TAM_MAX, 0);

            if (bytes_datos > 0 && strncmp(buffer, "DATOS|", 6) == 0) {
```

```
                char *datos_csv = buffer + 6;

                UPDATE peticion_upd;
                peticion_upd.num_tabla = tabla;
                peticion_upd.primary_key = llave_respaldo; // <-- USAMOS EL RESPALDO SEGURO
                peticion_upd.parametros = datos_csv;
                peticion_upd.error = NULL;

                solicitud_update(&peticion_upd, &directorio);

                if (peticion_upd.error != NULL) {
                    send(nuevo_socket, peticion_upd.error, strlen(peticion_upd.error), 0);
                    free(peticion_upd.error);
                }else {
                    // Si es NULL, significa que pasó todas las validaciones y se actualizó
                    char *msj_exito = "EXITO: El registro fue actualizado correctamente en el servidor.\n";
                    send(nuevo_socket, msj_exito, strlen(msj_exito), 0);
                }
            } else {
                 char *err = "ERROR: Se rompio el protocolo de actualizacion.\n";
                 send(nuevo_socket, err, strlen(err), 0);
            }
        }
    }
    // FINALIZACION DE SERVIDOR
    close(nuevo_socket);
    close(server_fd);
    printf("[*] Servidor apagado...\n");

    return 0;
}

// ==================================================================
// HILOS DE INICIALIZACION
// ==================================================================

void* funcion_h1(void *arg) {
    HILO_DIR *args = (HILO_DIR *)arg;
    printf("[Hilo DIR] Creando directorio de rutas para la base de datos...\n");
    crear_directorio(args->dir, args->carpeta, args->archivos, args->extension, args->update, args->tamano);
    return NULL;
}

```

```c
void* funcion_h2(void *arg) {
    HILO_RED *args = (HILO_RED *)arg;
    printf("[Hilo RED] Levantando servicio TCP en el puerto %d...\n", args->puerto);
    args->socket_fd_resultado = levantar_servicio(args->puerto);
    return NULL;
}
```

- **Archivos de Cabecera (.h)**

El archivo `entidades.h` actúa como el diccionario de datos principal y la interfaz base del sistema. Contiene la definición de todos los tipos de datos abstractos (estructuras) y los prototipos de las funciones fundamentales de bajo nivel que son compartidas por los distintos módulos del servidor.
En total, se definen 8 estructuras principales, categorizadas lógicamente según su propósito operativo:

- **Gestión de Sistema de Archivos (1):** La estructura `ARCHIVERO` es la encargada de almacenar dinámicamente en memoria las rutas absolutas de las tablas físicas y sus correspondientes archivos temporales (*buffers*) de actualización.
- **Control Transaccional CRUD (4):** Las estructuras principales son `CONSULTA`, `INSERCION`, `UPDATE` y `ELIMINACION`. Estas encapsulan los parámetros, las llaves primarias, los resultados y los búferes de error necesarios para ejecutar el ciclo de vida de los datos de manera aislada y segura.
- **Concurrencia e Inicialización (2):** Las estructuras `HILO_DIR` y `HILO_RED` están diseñadas específicamente para el empaquetado y transmisión limpia de parámetros hacia los hilos de ejecución (`pthreads`) durante el arranque paralelo del servidor.
- **Descubrimiento de Esquemas (1):** `ANALISIS_ARCHIVO` sirve como contenedor de metadatos espaciales para registrar el conteo dinámico de filas y columnas de cualquier tabla física evaluada.

Adicionalmente, este archivo de cabecera expone las firmas (prototipos) de cuatro funciones utilitarias críticas del núcleo: la inicialización del mapa de rutas (`crear_directorio`), la configuración e instanciación del socket TCP (`levantar_servicio`), el algoritmo de verificación de integridad referencial (`validar_llave`), y el analizador estructural de archivos (`analizar_archivo`).

Código 1. Servidor TCP (entidades.h)

```c
#ifndef ENTIDADES_H
#define ENTIDADES_H
#include <stdio.h>

// Estructuras complejas para la resolucion de las consultas, modificacion, insercion y
    eliminacion
// de los datos sobre las tablas
typedef struct
{
    int cantidad;
    char *rutas[14];
    char *rutas_update[14];
```

```c
} ARCHIVERO;

typedef struct
{
    int numero_tabla;            // indice de tablas estudiante = 0, direcciones[1] ...
    char *error;                 // Buffer de mensajes de error
    char *parametros;            // Máscara: "1" captura, "0" ignora
    char *llave;                 // Identificador único (llave primaria)
    int cantidad_resultados;     // Cuántos resultados se encontraron
    char **resultado;            // Array dinámico de strings con resultados
} CONSULTA;

typedef struct
{
    int numero_tabla;
    char *error;
    char *datos;
} INSERCION;

typedef struct
{
    int num_tabla;
    char *error;
    char *primary_key;
    char *parametros;
    void *estructura;
} UPDATE;

typedef struct {
    int num_tabla;
    char *error;
    char *primary_key;
} ELIMINACION;

typedef struct
{
    int num_lineas;
    int num_columnas;
} ANALISIS_ARCHIVO;

void crear_directorio(ARCHIVERO *dir, char *carpeta, char *archivos[], char *tipo, char
    *update, int cantidad_archivos);
int levantar_servicio(int puerto);
char *validar_llave(char *llave, int num_table, char *ruta);
ANALISIS_ARCHIVO analizar_archivo(char *ruta);

// Estructura del hilo 1 para ejecutar la funcion de crear directorio
typedef struct {
```

```
    ARCHIVERO *dir;
    char *carpeta;
    char *update;
    char **archivos;
    char *extension;
    int tamano;
} HILO_DIR;

// Estructura del hilo 2 para ejecuta la funcion de levantar servicio
typedef struct {
    int puerto;
    int socket_fd_resultado;
} HILO_RED;

#endif
```

- **Interfaces de Operación (`.h`)**

Los últimos tres archivos de cabecera (`consulta.h`, `insercion.h` y `update.h`) funcionan como las interfaces de programación (API) internas del motor. Su propósito es declarar de forma estricta las funciones que el servidor puede invocar para interactuar con los datos, aislando la lógica de red de la lógica de manipulación de archivos.

- **Interfaces de Operación (`.h`)**

Los últimos tres archivos de cabecera (`consulta.h`, `insercion.h` y `update.h`) funcionan como las interfaces de programación (API) internas del motor. Su propósito es declarar de forma estricta las funciones que el servidor puede invocar para interactuar con los datos, aislando la lógica de red de la lógica de manipulación de archivos.

- **`consulta.h`:** Define los prototipos necesarios para la recuperación de información. Incluye la función de alto nivel `solicitud_consulta`, que actúa como punto de entrada, y `consulta_tabla`, que ejecuta el escaneo físico y la aplicación de máscaras de bits sobre el archivo.
- **`insercion.h`:** Expone los métodos para la creación y erradicación de registros. Centraliza las funciones `solicitud_insercion`, que gestiona el flujo de nuevos datos, y `solicitud_eliminacion`, encargada de coordinar el proceso de filtrado y reemplazo de archivos para el borrado de información.
- **`update.h`:** Declara la interfaz para la modificación de registros existentes. Define la función `solicitud_update` para la validación previa de datos y `update_archivo` para la ejecución del algoritmo de reescritura mediante archivos temporales.

Esta arquitectura de cabeceras permite que el archivo principal del servidor (`servidor.c`) pueda ejecutar cualquier operación CRUD simplemente incluyendo la interfaz correspondiente, lo que facilita el mantenimiento y la escalabilidad del sistema ante futuras expansiones de las reglas de negocio.

Código 2. Servidor TCP (consulta.h)

```c
#ifndef CONSULTA_H
#define CONSULTA_H
#include "entidades.h"

void solicitud_consulta(CONSULTA *cliente, ARCHIVERO *dir);
void consulta_tabla(CONSULTA *cliente, ARCHIVERO *dir, int num_columnas);

#endif
```

Código 3. Servidor TCP (insercion.h)

```c
#ifndef INSERCION_H
#define INSERCION_H
#include "entidades.h"

void solicitud_insercion(INSERCION *cliente, ARCHIVERO *dir);
void solicitud_eliminacion(ELIMINACION *cliente, ARCHIVERO *dir);

#endif
```

- **Archivos fuente (.c)**

Este archivo (`servidor.c`) representa el nucleo del operativo de bajo nivel del servidor, tiene la logica fundacional que administra la memoria dinámica, el mapeo del sistema de archivos y la configuracion de red, sirviendo como soporte vital para la correcta ejecución de los módulos transaccionales (CRUD). Se compone de cuatro rutinas principales:

- **Géstion de Rutas y Memoria** (`crear_directorio`):
  Funcion encargada de la asignacion dinamica de memoria para estructurar los accesos a la base de datos fisica. Genera y almacena simultáneamente las rutas hacia el buffer (archivos temporales). Esta dualidad de rutas es la pieza clave que permite realizar las operaciones de actualización y eliminación de forma segura mediante el intercambio de archivos.

- **Descubrimiento Dinámico de Esquema** (`analizar_archivo`):
  Actúa como el analizador de metadatos del sistema. Mediante la lectura directa del archivo fisico `.csv`, calculando en tiempo real la cantidad de columnas (delimitadores) y el número total de filas. Esto dota de al servidor de una arquitectura agnóstica, permitiendo que el resto de las funciones adapten sus limites de memoria e iteraciones a cualquier tabla sin importar su tamaño, evitando la necesidad de constantes en el código.

- **Validación de integridad Referencial** (`validar_llave`):
  Es el escudo de seguridad de la base de datos para garantizar la unicidad de los registros. Apoyándose en el analizador de archivos, construye dinámicamente una máscara de bits para aislar exclusivamente la primera columna (Llave Primaria). Reutiliza el motor de consultas para escanear la tabla y determinar si un identificador ya está en uso (evitando sobreescritura) o si se encuentra libre.

- **Inicialización del Red TCP** (`inicilizar_servicio`):
  Abstrae la complejida de la pila de protocolos de comunicación. Se encarga de instanciar un socket bajo el protocolo IPv4 (`AF_INET`) y el protocolo orientado a conexión TCP (`SOCK_STREAM`). Configurar la reutilización de direcciones, enlaza el servicio a un puerto especifico (`bind`) y establece el estado de escucha activa (`listen`), retornando al hilo principal el descriptor de red listo para gestionar las peticiones de los clientes.

Código 1. Servidor TCP (funciones.h)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include "../extensiones/entidades.h"
#include "../extensiones/consulta.h"
#include "../extensiones/insercion.h"

extern ARCHIVERO directorio;

void crear_directorio(ARCHIVERO *dir, char *carpeta, char *archivos[], char *tipo, char
    *update, int cantidad_archivos)
{
    printf("Se esta creando un directorio con %d rutas de archivos %s\n",
    cantidad_archivos, tipo);

    for(int i = 0; i < cantidad_archivos; i++)
    {
        // 1. Calculamos la memoria exacta para cada cadena por separado
        size_t len_carpeta = strlen(carpeta) + strlen(archivos[i]) + strlen(tipo) + 1;
        size_t len_update  = strlen(update) + strlen(archivos[i]) + strlen(tipo) + 1;

        // 2. Asignamos la memoria correcta a cada puntero
        char *ruta_completa = (char *)malloc(len_carpeta);
        char *ruta_updates  = (char *)malloc(len_update);

        // Buena práctica: Verificar que malloc no falle
        if (ruta_completa == NULL || ruta_updates == NULL) {
            printf("ERROR: Memoria insuficiente para crear rutas.\n");
            return;
        }

        // 3. Construimos la ruta principal (ej. "./Datos/Estudiante.csv")
        strcpy(ruta_completa, carpeta);
        strcat(ruta_completa, archivos[i]);
        strcat(ruta_completa, tipo);

```

```
        // 4. Construimos la ruta temporal (ej. "./buffer/Estudiante.csv")
        strcpy(ruta_updates, update);
        strcat(ruta_updates, archivos[i]);
        strcat(ruta_updates, tipo);

        // 5. Guardamos en el archivero
        dir->rutas[i] = ruta_completa;
        dir->rutas_update[i] = ruta_updates;
        dir->cantidad = i + 1;
    }
}

ANALISIS_ARCHIVO analizar_archivo(char *ruta)
{
    ANALISIS_ARCHIVO analisis = {0, 0};
    FILE *archivo = fopen(ruta, "r");

    char linea[512];
    int primera_linea = 1;
    while (fgets(linea, sizeof(linea), archivo) != NULL) {
        analisis.num_lineas++;
        if (primera_linea) {
            // Contar columnas: número de comas + 1
            char *ptr = linea;
            while (*ptr) {
                if (*ptr == ',') {
                    analisis.num_columnas++;
                }
                ptr++;
            }
            analisis.num_columnas++; // Contar ultimo renglon por sintaxis .csv
            primera_linea = 0;
        }
    }
    // Cerramos el archivo y regresamos el analisis
    fclose(archivo);
    return analisis;
}

char *validar_llave(char *llave, int num_table, char *ruta)
{
    // 1. Analizar el archivo para saber cuántas columnas reales tiene
    ANALISIS_ARCHIVO analisis = analizar_archivo(ruta);
    int columnas = analisis.num_columnas;

    // 2. Construir la máscara dinámica (ej. "10000" para 5 columnas)
    // Reservamos memoria para las columnas + el carácter nulo '\0'
    char *mascara = malloc(columnas + 1);
```

```
    if (mascara == NULL) return "ERROR: Memoria insuficiente";

    mascara[0] = '1'; // Activamos solo la primera columna (la llave)
    for (int i = 1; i < columnas; i++) {
        mascara[i] = '0'; // Apagamos el resto de las columnas
    }
    mascara[columnas] = '\0'; // Cerramos la cadena de C

    CONSULTA tmp;
    tmp.numero_tabla = num_table;
    tmp.error = NULL;
    tmp.llave = llave;
    tmp.cantidad_resultados = 0;
    tmp.resultado = NULL;
    tmp.parametros = mascara; // Asignamos nuestra cadena recién creada

    // Lanzamos la consulta pasándole el número real de columnas
    consulta_tabla(&tmp, &directorio, columnas);

    // 3. LIMPIEZA: Ya usamos la máscara, debemos liberarla
    free(mascara);

    // 4. Verificamos si la consulta encontró la llave
    if (tmp.cantidad_resultados > 0)
    {
        // Liberamos la memoria de los resultados
        for (int i = 0; i < tmp.cantidad_resultados; i++) {
            free(tmp.resultado[i]);
        }
        free(tmp.resultado);

        return "ERROR: La llave se repite en la tabla\n";
    }

    return "EXITO";
}

int levantar_servicio(int puerto)
{
    int servidor_fd;
    struct sockaddr_in direct;
    int opt = 1;

    // 1. Crear el socket: AF_INET (IPv4), SOCK_STREAM (TCP)
    if ((servidor_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("ERROR: Fallo al crear el socket TCP");
        exit(EXIT_FAILURE);
    }
```

```

    // Opcional pero recomendado: Permite reutilizar el puerto inmediatamente
    // si reinicias el servidor rápido durante tus pruebas
    setsockopt(servidor_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // 2. Configurar la estructura de la dirección
    bzero(&direct, sizeof(direct));
    direct.sin_family = AF_INET;
    direct.sin_addr.s_addr = INADDR_ANY; // Escucha en cualquier interfaz de red
    direct.sin_port = htons(puerto);

    // 3. Vincular el socket al puerto (bind)
    if (bind(servidor_fd, (struct sockaddr *)&direct, sizeof(direct)) < 0) {
        perror("ERROR: Fallo en bind (Puerto en uso)");
        exit(EXIT_FAILURE);
    }

    // 4. Poner el socket en modo escucha (listen) - Exclusivo de TCP
    // El '5' indica cuántas conexiones pueden esperar en la cola antes de ser
   rechazadas
    if (listen(servidor_fd, 5) < 0) {
        perror("ERROR: Fallo en listen");
        exit(EXIT_FAILURE);
    }

    printf("\n[+] Servicio TCP levantado. Escuchando en el puerto %d...\n", puerto);

    // Retornamos el descriptor para que el main pueda aceptar a los clientes
    return servidor_fd;
}
```

Este archivo (consulta.c / consulta.h) es el módulo encargado de gestionar las operaciones de lectura sobre la base de datos. Como primer paso de seguridad, valida que los parámetros enviados por el cliente coincidan con la estructura física del archivo. De detectar alguna anomalía, intercepta la petición y retorna un mensaje de error descriptivo, lo que permite al usuario corregir su formato sin poner en riesgo la integridad del sistema.
Adicionalmente, el motor cuenta con la lógica necesaria para procesar tres niveles de consulta:

- **Completa:** Extrae y retorna todos los registros de la tabla con la totalidad de sus columnas.
- **Parcial:** Recorre todos los registros del archivo, pero filtra y devuelve unicamente las columnas que el cliente seleccionó mediante la mascara de bits.
- **Unica:** Localiza un registro en especifico mediante su llave primaria y extra exclusivamente las columnas que el usuario desea observar de dicho registro.

Código 2. Servidor TCP (consulta.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include "../extensiones/entidades.h"
#include "../extensiones/consulta.h"

extern int TAM_MAX;

void procesar_consulta(int socket_cliente, int tabla, char *llave, char *mascara,
    ARCHIVERO *dir) {
    if (llave == NULL || mascara == NULL) {
        char *msj = "ERROR: Faltan parametros en la consulta.\n";
        send(socket_cliente, msj, strlen(msj), 0);
        return;
    }

    CONSULTA c = {tabla, NULL, mascara, llave, 0, NULL};
    solicitud_consulta(&c, dir);

    // 1. Filtro de Errores vs Exito
    if (c.error != NULL) {
        if (strncmp(c.error, "ERROR", 5) == 0) {
            send(socket_cliente, c.error, strlen(c.error), 0);
            free(c.error);
            return; // Terminamos aquí si hubo error
        }
        free(c.error);
    }

    // 2. Respuesta Vacía
    if (c.cantidad_resultados == 0) {
        char *msj_vacio = "No se encontraron registros coincidentes.\n";
        send(socket_cliente, msj_vacio, strlen(msj_vacio), 0);
        return;
    }

    // 3. Protocolo Ping-Pong
    if (c.cantidad_resultados > 0) {
        char metadatos[50];
        char buffer_ack[TAM_MAX]; // Buffer local solo para los ACKs

        sprintf(metadatos, "CANTIDAD|%d", c.cantidad_resultados);
        send(socket_cliente, metadatos, strlen(metadatos), 0);

```

```
        bzero(buffer_ack, TAM_MAX);
        recv(socket_cliente, buffer_ack, TAM_MAX, 0); // Esperamos 1er ACK

        for (int i = 0; i < c.cantidad_resultados; i++) {
            send(socket_cliente, c.resultado[i], strlen(c.resultado[i]), 0);

            bzero(buffer_ack, TAM_MAX);
            recv(socket_cliente, buffer_ack, TAM_MAX, 0); // Esperamos siguiente ACK

            free(c.resultado[i]);
        }
        free(c.resultado);
    }
}

void solicitud_consulta(CONSULTA *cliente, ARCHIVERO *dir)
{
    // 1. Validar que el número de tabla esté dentro de los límites
    if (cliente->numero_tabla < 0 || cliente->numero_tabla >= dir->cantidad) {
        cliente->error = strdup("ERROR: LA TABLA SOLICITADA NO EXISTE\n");
        return;
    }

    // 2. Descubrimiento de Esquema
    char *ruta_archivo = dir->rutas[cliente->numero_tabla];
    ANALISIS_ARCHIVO analisis = analizar_archivo(ruta_archivo);
    int columnas_reales = analisis.num_columnas;

    printf("--------||| Se realiza consulta en la tabla %d |||--------\n", cliente->
    numero_tabla);

    // 3. Verificamos que la máscara de bits coincida con las columnas reales
    if (strlen(cliente->parametros) != columnas_reales) {
        char buffer_error[256];
        sprintf(buffer_error, "ERROR: La mascara ingresada no coincide. Esta tabla
    requiere exactamente %d parametros (Bits 1/0).\n", columnas_reales);
        cliente->error = strdup(buffer_error);
    }
    // 4. Si todo está perfecto, lanzamos la consulta física
    else {
        consulta_tabla(cliente, dir, columnas_reales);
    }
}

void consulta_tabla(CONSULTA *cliente, ARCHIVERO *dir, int num_columnas)
{
    ANALISIS_ARCHIVO analisis = analizar_archivo(dir->rutas[cliente->numero_tabla]);

```

```
    FILE *tabla = fopen(dir->rutas[cliente->numero_tabla], "r");

    char linea[300];
    int hallados = 0;
    cliente->resultado = malloc(sizeof(char*) * analisis.num_lineas);
    while (fgets(linea, sizeof(linea), tabla) != NULL) {
        linea[strcspn(linea, "\n")] = '\0';

        char *tokens[15];
        int col = 0;
        char *token = strtok(linea, ",");
        while (token != NULL && col < num_columnas) {
            tokens[col++] = token;
            token = strtok(NULL, ",");
        }

        if (col == 0) {
            continue;
        }

        int es_full = (cliente->llave != NULL && strcasecmp(cliente->llave, "full") ==
0);
        int es_low = (cliente->llave != NULL && strcasecmp(cliente->llave, "low") == 0);
        int match = 0;

        if (es_full || es_low) {
            match = 1;
        } else if (cliente->llave == NULL) {
            match = 1;
        } else if (tokens[0] != NULL && strcmp(tokens[0], cliente->llave) == 0) {
            match = 1;
        }

        if (!match) {
            continue;
        }

        char buffer[1024] = "";
        int primero = 1;

        if (es_full) {
            for (int i = 0; i < col; i++) {
                if (!primero) strcat(buffer, ",");
                strcat(buffer, tokens[i]);
                primero = 0;
            }
        } else {
            for (int i = 0; i < num_columnas && i < col; i++) {
```

```
                if (cliente->parametros[i] == '1') {
                    if (!primero) strcat(buffer, ",");
                    strcat(buffer, tokens[i]);
                    primero = 0;
                }
            }
        }
        cliente->resultado[hallados++] = strdup(buffer);
    }
    cliente->cantidad_resultados = hallados;
    // Liberamos la memoria no ocupada
    if (hallados > 0 && hallados < analisis.num_lineas) {
        cliente->resultado = realloc(cliente->resultado, sizeof(char*) * hallados);
    }
    if (hallados == 0) {
        free(cliente->resultado);
        cliente->resultado = NULL;
        cliente->error = strdup("ERROR: No se encontro ningun registro en la tabla
    consultada\n");
    }else{
        cliente->error = strdup("EXITO: Se encontro al menos 1 registro en la tabla\n");
    }

    fclose(tabla);
}
```

El siguiente archivo (insercion.h) concentra las funciones encargadas de alterar la estructura física de los datos almacenados, garantizando en todo momento la integridad referencial y estructural de los archivos de texto secuenciales (.csv). Se divide en dos operaciones fundamentales: inserción y eliminación.

- **Operacion de inserción**
  Esta función gestiona la agregación de nuevos registros a la base de datos aplicando un modelo de validación estricto antes de realizar cualquier escritura en disco. Su flujo de ejecución consiste en:

  1. **Validación de Integridad Estructural:** Analiza la cadena de datos entrante para asegurar que no exstan campos nulos o vacios (detectando las comas contiguas)
  2. **Extracción y Validación de llave primaria:** Realiza una copia en memoria de la cadena original para extraer la llave primaria sin corromper el paquete de datos. Posteriormente, verifica mediante el motor de busqueda que dicha llave está completamente libre; si detecta un colision (duplicidad), la operación se aborta para proteger la unicidad de los registros.
  3. **Escritura Fisica:** Una vez superados los filtros de seguridad, el sistema abre el archivo en modo de escritura e inserta la nueva fila de manera limpia al final del documento, cerrando el flujo de datos para confirmar la persistencia.

- **Operacion de Eliminación**
  La función implementa un mecanismo de borrado logico y fisico utilizando una arquitectura de archivos temporales (buffer) para evitar la corrupción de datos. Su proceso se define en los siguiente pasos:

1. **Comprobación de Existencia:** Antes de iniciar cualquier manipulación de archivos, verifica que la llave primaria solicitada realmente exista en el sistema, evitando procesar ”Registros fantasmas”
2. **Tránsito por Buffer Temporal:** El sistema abre simultáneamente el archivo original en modo lectura y una archivo temporal en modo escritura.
3. **Filtrado y Reemplazo:** El motor lee el archivo original linea por linea. Cuando detecta la fila que coincide con la llave primaria a eliminar, la ignora deliberadamente(omite su escritura) y el resto de los registros se copian intactos al archivo temporal.
4. **Persistencia Final:** Al terminar el ciclo, los flujos de memoria se cierran y elimina el archivo original fisicamente, para que el archivo temporal se renombre al archivo antiguo tomando asi su lugar, permitiendo asi la eliminacion del registro.

Código 3. Servidor TCP (insercion.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "../extensiones/entidades.h"
#include "../extensiones/insercion.h"

void solicitud_insercion(INSERCION *cliente, ARCHIVERO *dir)
{
    // 1. Verificamos que la tabla sea válida
    if (cliente->numero_tabla < 0 || cliente->numero_tabla > 13) {
        cliente->error = strdup("ERROR: Tabla no valida para insercion");
        return;
    }

    // 2. Validar campos vacíos rápidamente
    // Buscamos si hay dos comas juntas (,,) o si empieza/termina con coma
    if (strstr(cliente->datos, ",,") != NULL ||
        cliente->datos[0] == ',' ||
        cliente->datos[strlen(cliente->datos) - 1] == ',')
    {
        cliente->error = strdup("ERROR: Los datos contienen campos vacios.");
        return;
    }

    // 3. Extraer la llave primaria de los datos para validarla
    char datos_copia[1024];
    strcpy(datos_copia, cliente->datos);
    char *llave_nueva = strtok(datos_copia, ",");

    if (llave_nueva == NULL) {
        cliente->error = strdup("ERROR: Formato de datos invalido.");
        return;
```

```
    }

    char *ruta_archivo = dir->rutas[cliente->numero_tabla];

    // 4. Validamos que la llave NO exista (Para INSERT, "EXITO" significa que la llave está libre)
    char *estado = validar_llave(llave_nueva, cliente->numero_tabla, ruta_archivo);

    if (strcmp(estado, "EXITO") != 0) {
        // Si no es EXITO, es porque validar_llave encontró duplicados o hubo un error
        cliente->error = strdup("ERROR: La llave primaria ya existe en la base de datos.");
        return;
    }

    // 5. Procedemos a insertar al final del archivo (Modo "a" - Append)
    FILE *archivo = fopen(ruta_archivo, "a");
    if (archivo == NULL) {
        cliente->error = strdup("ERROR: No se pudo abrir el archivo para insertar.");
        return;
    }

    // Escribimos la línea completa directamente
    fprintf(archivo, "%s\n", cliente->datos);
    fclose(archivo);

    cliente->error = strdup("EXITO: Registro insertado correctamente.");
}

void solicitud_eliminacion(ELIMINACION *cliente, ARCHIVERO *dir)
{
    // 1. Validamos que el registro SI EXISTA
    // Recordamos que si validar_llave retorna "EXITO", significa que la llave está libre (No existe)
    char *estado = validar_llave(cliente->primary_key, cliente->num_tabla, dir->rutas[cliente->num_tabla]);
    if(strcmp(estado, "EXITO") == 0)
    {
        cliente->error = strdup("ERROR: La llave no existe. No se puede eliminar un registro fantasma.");
        return;
    }

    // 2. Rutas
    char *ruta_original = dir->rutas[cliente->num_tabla];
    char *ruta_temporal = dir->rutas_update[cliente->num_tabla]; // Usamos la misma carpeta ./buffer/

```

```c
    FILE *archivo_lectura = fopen(ruta_original, "r");
    FILE *archivo_escritura = fopen(ruta_temporal, "w");

    if (!archivo_lectura || !archivo_escritura) {
        cliente->error = strdup("ERROR: Fallo critico al abrir los archivos para eliminacion.");
        if(archivo_lectura) fclose(archivo_lectura);
        if(archivo_escritura) fclose(archivo_escritura);
        return;
    }

    char linea[1024];

    // 3. Procesamiento de líneas
    while (fgets(linea, sizeof(linea), archivo_lectura)) {

        char linea_copia[1024];
        strcpy(linea_copia, linea);
        char *llave_actual = strtok(linea_copia, ",");

        // 4. Si encuentra la llave hace un salto de linea y la "elimina"
        if (llave_actual != NULL && strcmp(llave_actual, cliente->primary_key) == 0) {
            // ENCONTRAMOS EL REGISTRO
            // Para eliminarlo, simplemente usamos 'continue' para saltarnos la escritura
            // y pasar a la siguiente línea del while.
            continue;
        } else {
            // Si no es la llave que buscamos, lo copiamos al archivo temporal
            fputs(linea, archivo_escritura);
        }
    }

    // 5. Limpieza y reemplazo
    fclose(archivo_lectura);
    fclose(archivo_escritura);

    remove(ruta_original);

    if (rename(ruta_temporal, ruta_original) == 0) {
        cliente->error = strdup("EXITO: Registro eliminado permanentemente.");
    } else {
        cliente->error = strdup("ERROR: No se pudo completar la eliminacion fisica.");
    }
}
```

Por ultimo el archivo (`update.c`) concentra la logica de transaccional encargada de registros exsistentes. Su diseño divide el proceso en dos fases criticas para garantizar la integridad referencial y prevenir la

corrupción de la base de datos durante las operaciones de escritura.

- **Fase 1:** Validación y Prevención de Colisiones (`solicitud_update`)
  Antes de interactuar con los archivos fisicos, está función actúa como un filtro estricto de regla de negocio:

  1. **Comprobación de Existencia:** Verifica mediante el analizador de llaves que el registro original que se desea modificar efectivamente existen en el sistema.
  2. **Extracción Segura:** Realiza una copia profunda `deep copy` de la cadena de la cadena de nuevos parametros para extraer la nueva llave primaria sin alterar el paquete de datos original.
  3. **Control de Alteración de Llave:** Si el sistema detecta que la actualización incluye un cambio de nueva llave primaria (es decir, la llave nueva es diferente a la original), lanza una validación secundaria. Está comprobación garantiza que la nueva llave está completamente libre. Si la llave de destino ya está en uso por otro registro, la transacción se aborta inmediatamente para evitar duplicidad y proteger la unicidad de las entidades.

- **Fase 2:** Ejecución y Persistencia Física (`update_archivo`)
  Una vez que los datos son catalogados como seguros, el sistema procede con la sobreescritura utilizando una estrategia de buffer temporal:

  1. **Apertura Dual:** Se establece un flujo de lectura desde el archivo original (en `./Datos/`) y un flujo de escritura hacia un nuevo archivo temporal (en `./buffer/`).
  2. **Reemplazo Condicional:** El motor lee el archivo original linea por linea. Cuando el curso detecta la fila cuya llave primaria coincide con la del registro a modificar, el sistema omite la linea y, en su lugar, inyecta la nueva cadena parametros o datos proporcionados por el cliente. El resto de los registros se transcribe de forma idéntica.
  3. **Consolidación (commit físico):** Al finalizar la lectura, los flujos de memoria se liberan. El sistema operativo elimina el archivo `.csv` original y reubica el archivo temporal en el directorio principal, renombrandolo como el original para que tome el lugar de su predecesor, completando así la transacción de manera atómica.

Código 4. Servidor TCP (update.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include "../extensiones/entidades.h"
#include "../extensiones/update.h"

void solicitud_update(UPDATE *cliente, ARCHIVERO *dir)
{
    // 1. Validamos que el registro ORIGINAL exista
    char *estado_origen = validar_llave(cliente->primary_key, cliente->num_tabla, dir->
   rutas[cliente->num_tabla]);
    if(strcmp(estado_origen, "EXITO") == 0)
```

```
    {
        cliente->error = strdup("ERROR: El registro que intentas actualizar no existe.\n");
        return;
    }

    // 2. Extraemos la NUEVA llave de forma segura (sin destruir cliente->parametros)
    char parametros_copia[500];
    strcpy(parametros_copia, cliente->parametros);
    char *nueva_llave = strtok(parametros_copia, ",");

    // 3. Verificamos si el usuario está intentando CAMBIAR la llave primaria
    if (nueva_llave != NULL && strcmp(cliente->primary_key, nueva_llave) != 0)
    {
        // Como son diferentes, validamos que la nueva llave NO esté ocupada
        char *estado_destino = validar_llave(nueva_llave, cliente->num_tabla, dir->rutas[cliente->num_tabla]);

        // Si NO retorna "EXITO", significa que la llave ya la tiene otro registro (Duplicado)
        if (strcmp(estado_destino, "EXITO") != 0)
        {
            cliente->error = strdup("ERROR: La nueva llave primaria ya esta en uso por otro registro.\n");
            return;
        }
    }

    // 4. Si llegamos aquí, todo es válido (la llave es la misma o la nueva está libre)
    cliente->error = NULL;
    update_archivo(cliente, dir);
}

void update_archivo(UPDATE *cliente, ARCHIVERO *dir)
{
    char *ruta_original = dir->rutas[cliente->num_tabla];
    char *ruta_temporal = dir->rutas_update[cliente->num_tabla];

    FILE *archivo_lectura = fopen(ruta_original, "r");

    // [MOMENTO 1: NACIMIENTO]
    // Aquí el sistema operativo CREA el archivo físicamente en la carpeta ./buffer/
    FILE *archivo_escritura = fopen(ruta_temporal, "w");

    if (!archivo_lectura || !archivo_escritura) {
        cliente->error = strdup("ERROR: Fallo critico al abrir/crear los archivos.\n");
        if(archivo_lectura) fclose(archivo_lectura);
        if(archivo_escritura) fclose(archivo_escritura);
```

```c
        return;
    }

    // [MOMENTO 2: CRECIMIENTO]
    // Copiamos datos y metemos el registro actualizado
    char linea[1024];
    while (fgets(linea, sizeof(linea), archivo_lectura)) {
        char linea_copia[1024];
        strcpy(linea_copia, linea);
        char *llave_actual = strtok(linea_copia, ",");

        if (llave_actual != NULL && strcmp(llave_actual, cliente->primary_key) == 0) {
            fprintf(archivo_escritura, "%s\n", cliente->parametros);
        } else {
            fputs(linea, archivo_escritura);
        }
    }

    // Cerramos para liberar la memoria RAM
    fclose(archivo_lectura);
    fclose(archivo_escritura);

    // [MOMENTO 3: DESTRUCCION DEL VIEJO]
    // Borramos el archivo original de la carpeta ./Datos/
    remove(ruta_original);

    // [MOMENTO 4: REENCARNACION]
    // Movemos el archivo temporal de ./buffer/ hacia ./Datos/ y le cambiamos el nombre
    if (rename(ruta_temporal, ruta_original) != 0){
        cliente->error = strdup("ERROR: No se pudo mover el archivo temporal.\n");
        return;
    }
    printf("EXITO: Se actualizo el registro %s\n", cliente->primary_key);
}
```

Conclusiónes

Referencias

- CRUD - Glosario de MDN Web Docs — MDN. (s. f.). `https://developer.mozilla.org/es/docs/Glossary/CRUD`
- Amazon Web Services. (s. f.). ¿Qué es un cliente ligero? Amazon Web Services, Inc. `https://aws.amazon.com/es/what-is/thin-client/`
- Amazon Web Services. (s. f.-b). Qué es una CLI (interfaz de la línea de comandos)? Amazon Web Services, Inc. Amazon Web Services. (s. f.-b). Qué es una CLI (interfaz de la línea de comandos)?

Amazon Web Services, Inc. `https://aws.amazon.com/es/what-is/cli`